Detection Copilot: abertura do projeto

Treinando um modelo de IA local para escrever regras de detecção que funcionam de verdade

Autor: alefkaue · Início: 08/11/2026 · Status: Fase 0 (fundação)

---

1. O problema

Escrever regras de detecção é um dos trabalhos mais valiosos e mais demorados de um time de segurança. Quando sai um relatório sobre uma nova técnica de ataque, alguém precisa transformar aquele texto numa regra que o SIEM entenda, e testar se ela pega o ataque sem disparar alarme falso.

Modelos de IA genéricos ajudam pouco: geram regras com campos errados, sintaxe inválida ou que simplesmente não disparam. Um estudo recente mostrou que LLMs sem estrutura deixaram passar **todos** os alertas maliciosos; com guias e validação, chegaram a 93%.

2. A proposta

Um **copiloto de detection engineering** que roda **100% local**: os dados não saem da máquina e não há custo de API.

1. Recebe a descrição de um comportamento malicioso.
2. Um modelo **Qwen de 4 bilhões de parâmetros**, treinado com LoRA para essa tarefa, gera uma **regra Sigma**, o formato aberto aceito por Splunk, Elastic, Sentinel e outros.
3. Um **juiz automático** testa a regra antes de entregá-la.

3. Como o sucesso é medido

Cada regra gerada passa por três testes:

Teste	Pergunta	Ferramenta
Sintaxe	É uma regra Sigma válida?	pySigma
Detecção	Dispara no log real do ataque?	Zircolite + logs `.evtx` de ataques conhecidos
Precisão	Fica quieta em logs normais?	Zircolite + logs de uma VM de laboratório

Métrica principal: porcentagem de regras que passam nos três testes. Ela é medida no modelo **sem treino** (linha de base) e depois de **cada rodada de treino**, e esse antes × depois é o resultado central do projeto.

4. Roteiro

Fase	Entrega	Duração
0. Fundação	Repositório, ambiente, primeira detecção de um ataque real	2–3 dias
1. Dataset	Pares "descrição → regra" (SigmaHQ), com treino e teste separados por técnica ATT&CK	1–2 semanas
2. Juiz + linha de base	Pipeline de validação; medição do modelo sem treino	1 semana
3. Primeiro treino	LoRA no Google Colab; importação no Ollama; nova medição	1–2 semanas
4. Análise de erros	Onde o modelo erra e como melhorar os dados	1–2 semanas
5. Produto	Servidor MCP + CLI: o analista cola um relatório e recebe a regra testada	1–2 semanas
6. Lab ao vivo	VM Windows + Sysmon + ataque simulado (Atomic Red Team) detectado pela regra gerada	1–2 semanas

5. Tecnologias

Área	Escolha
Modelo	Qwen3.5-4B via Ollama (local, ~65 tokens/s numa GPU AMD de 8 GB)
Treino	Unsloth + LoRA bf16 no Google Colab (~10 GB de VRAM), exportado para GGUF
Regras	Sigma (SigmaHQ como referência)
Validação	pySigma (sintaxe) e Zircolite (execução sobre `.evtx`)
Dados de ataque	EVTX-ATTACK-SAMPLES, amostras do Hayabusa e, depois, logs gerados em laboratório
Linguagem	Python 3.12

6. Como o trabalho é feito

O projeto usa assistentes de IA de forma **declarada e delimitada**:

Quem	Papel
Eu	Decisões; escrevo o núcleo (juiz de validação, dataset, treino, análise de erros); experimentos; textos públicos
Claude Code	Mentor e arquiteto: planeja, explica, guia passo a passo, revisa meu código
Codex	Tarefas de apoio delimitadas (coleta de dados, testes, CI) e segunda revisão; assume a mentoria quando o limite do Claude acaba

As regras ficam num arquivo único (`AGENTS.md`) lido pelos dois agentes, e um quadro de tarefas com passagem de bastão garante que um continue de onde o outro parou. Nenhum código entra no repositório sem ter sido lido e entendido por mim.

7. Riscos e cuidados

Risco	Mitigação
Treino e teste "vazarem" (o modelo decorar regras parecidas)	Separar o dataset por técnica ATT&CK, não aleatoriamente
Falso positivo difícil de medir	Gerar logs normais numa VM de laboratório
Privacidade (projeto público de um profissional de segurança)	Logs só de laboratório; verificador automático de dados pessoais antes de publicar
Licença das regras	SigmaHQ sob Detection Rule License: atribuição no repositório e no modelo publicado
Qualidade após exportar o modelo	Manter o mesmo chat template e o mesmo token de fim no GGUF

8. Referências principais

- Cai et al., *From Texts to Rules: Generating Sigma Rules with LLMs from Cyber Threat Reports*, USENIX Security 2026.
- *OpenSOC-AI: Parameter Efficient LLM Log Analysis*, arXiv 2604.26217.
- *Fine-tuning de modelos pequenos em logs rotulados com ATT&CK*, arXiv 2606.18190.
- Unsloth: guia oficial de fine-tuning do Qwen3.5.
- Projetos SigmaHQ, Zircolite e EVTX-ATTACK-SAMPLES.

Lista completa e comentada em `docs/pesquisa/fontes.md`.